

Built for *Humans*

Why the Founders Who Win Y Combinator
Build for People, Not Portfolios

Sreekesari Vangeepuram

License

CC BY-NC-SA 4.0

This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

You are free to share and adapt this material for non-commercial purposes, provided you give appropriate credit and distribute any derivative works under the same license terms.

For commercial licensing inquiries, please contact the author.

builtforhumans@proton.me

creativecommons.org/licenses/by-nc-sa/4.0



The very best startup ideas tend to have three things in common: they're something the founders themselves want, that they themselves can build, and that few others realize are worth doing.

Paul Graham, Y Combinator Co-Founder (2005)

Contents

Preface: A Confession	6
PART I: THE MINDSET	7
1. The Humility Imperative	8
2. What Y Combinator Actually Funds	11
3. The AI-Native Distinction	14
PART II: THE PROBLEM	17
4. Finding a Problem Worth Solving	18
5. Customer Discovery for Builders	21
6. The Problem Thesis	24
PART III: THE BUILD	26
7. The Multi-Agent Workforce	27
8. Architecture for Speed	31
9. From Code to Customers	34
PART IV: THE PITCH	36
10. The Y Combinator Application	37
11. The Interview and Beyond	40
PART V: THE REALITY	42
12. The First Hundred Days	43
13. The Founder's Discipline	45
APPENDICES	48
A. The Six-Week Execution Checklist	48
B. MAS Prompt Library	49
C. YC Application Field Reference	50
D. Idea Evaluation Matrix	50
References	51

Preface: A Confession

Let me tell you about the founders who do not get into Y Combinator. They are brilliant. They have built large language model orchestration systems that would make a computer science professor pause. They can explain transformer architecture at a coffee shop. They have GitHub profiles that glow with contribution activity. And they have built products that precisely zero people want to use.

The rejection email stings because it feels unfair. They *are* talented. They *did* work hard. But talent and effort were never the point. Y Combinator partners are not evaluating intelligence. They are evaluating obsession with a specific human problem. And that obsession can only be demonstrated through contact with the humans who have the problem.

In every application cycle, Y Combinator receives over 10,000 applications and accepts roughly 1%^[1]. The vast majority of rejections happen at the written application stage, before any interview takes place. The applications that win have one thing in common: the founders talked to users before building. Everything else is secondary.

This is not a book about how to impress Y Combinator. It is a book about how to become the kind of founder YC cannot ignore: one who builds for humans first and lets the technology serve that mission. If you are reading this because you want to build an AI-native product and get into the most competitive startup accelerator in the world, the path is simpler than you think and harder than you want. It requires talking to strangers before you feel ready. Shipping things that are embarrassingly incomplete. Charging money before you feel worthy. And never, ever, falling in love with your own cleverness.

WHAT THIS BOOK ASSUMES

This book assumes you have technical ability or access to it through AI agents. It assumes you are willing to work harder than you thought possible for six weeks. It assumes you care more about solving a real problem than about building a perfect product. If those assumptions are wrong, this book is not for you.

PART I

The Mindset

Before you build a single thing

1. The Humility Imperative

"The very best startup ideas tend to have three things in common: they're something the founders themselves want, that they themselves can build, and that few others realize are worth doing."

Paul Graham, *How to Get Startup Ideas* (2012)

The most common mistake technically gifted founders make is building a monument to their own intelligence rather than a bridge to someone else's problem. They treat their startup like a portfolio piece that demonstrates their capabilities. The product must use the latest framework, the most elegant architecture, the cutting-edge model. It must impress other engineers. It must be defensible at dinner parties. This mindset is fatal because it inverts the founding equation.

The Portfolio Trap

Every year, thousands of technically gifted engineers enter Y Combinator applications with what experienced founders call the *portfolio mindset*. They treat their startup like a piece of art that demonstrates their capabilities. The product must use the latest framework, the most elegant architecture, the cutting-edge model. It must impress other engineers. It must be defensible at dinner parties.

This mindset is fatal because it inverts the founding equation. You are no longer asking "What does the user need?" You are asking "What makes me look capable?" The product becomes a mirror reflecting your skills rather than a bridge solving someone's pain.

Paul Graham addressed this directly in his widely cited essay on startup ideas: the best ideas are ones where the founders themselves experienced the problem, can build the solution, and few others realize the opportunity exists^[2]. The portfolio mindset violates all three principles. It builds for an audience of peers rather than an audience of users.

Here is a practical test: if you cannot describe your product without mentioning the technology stack, you are building for your portfolio. If your pitch starts with "We use a multi-agent system with retrieval-augmented generation" instead of "We save medical billers six hours every day," you have already lost the attention of anyone who matters.

The Three Questions Every User Asks

When someone encounters your product, they are not evaluating your technical sophistication. They are asking three questions, usually subconsciously, always brutally:

What is this? Not "What technology powers this?" but "What category of thing is this, and do I need it?" Research on consumer decision-making confirms that users form an impression within 50 milliseconds of viewing a webpage^[3]. You have three seconds. If your landing page requires understanding what a "multi-agent orchestration framework" is, you have already lost 99% of visitors.

Will it work for me? Users are evaluating fit, not capability. They want to see their specific situation reflected in your product. A medical biller wants to see a medical billing interface, not a generic AI playground. Specificity creates trust faster than sophistication. This insight is supported by decades of work in user-centered design, where contextual appropriateness consistently outperforms feature richness in driving adoption^[4].

What happens if I trust you? Every new product requires a leap of faith. The user must believe that investing their time and data in your product will produce a meaningful return. Testimonials, specific metrics, and clear value propositions reduce this risk. Technical elegance does not. Rob Fitzpatrick captured this in *The Mom Test*, where he demonstrated that the only validation that matters is concrete commitment from users, not polite encouragement^[5].

THE HUMILITY CHECK

Before writing your first line of code, write one sentence answering this: "My product helps [specific person] do [specific thing] without [specific pain]." If you cannot fill in all three blanks with concrete specificity, you are not ready to build. You are ready to learn.

Building for People Means Letting Go

The most difficult transition for technical founders is accepting that users do not care about the internal beauty of their systems. A user cannot see your elegant agent communication protocol. They cannot appreciate your sophisticated retrieval architecture. They can only see the interface, the output, and the result in their workflow.

This means making painful tradeoffs. It means shipping a frontend that embarrasses your design sensibilities because the user needs the feature on Tuesday. It means using a simpler model because the latency improvement matters more than the accuracy gain. It means hardcoding a solution for your first ten customers rather than building a generalizable system.

These tradeoffs feel like compromises. They are not. They are the definition of product judgment: the ability to correctly weigh what matters to the user against what matters to your engineering pride. As Steve Blank argued in *The Four Steps to the Epiphany*, startups succeed not through perfect execution of a plan, but through rapid learning cycles driven by customer feedback^[6].

The founders who win are not the ones with the cleanest code. They are the ones who held their ego loosely enough to let the user reshape their vision. They built something imperfect for someone specific, then iterated based on real feedback rather than theoretical purity.

THE FOUNDER WHO LEARNED

A founder spent three months building a perfectly architected AI system for automated code review. The system used seven different agents, a custom vector database, and a novel consensus algorithm. He applied to YC with impressive technical documentation and zero users. Rejected.

He spent the next week calling engineering managers. Not pitching. Just asking about their code review process. He discovered that the real pain was not in the review itself but in the *assignment* of reviewers: figuring out who should review what. He built a dead-simple matching tool in three days using a single GPT-4 call. No fancy architecture. No custom database. Just a form that took a pull request description and suggested reviewers based on past contributions.

It was ugly. It worked. Within two weeks, three teams were using it. He reapplied with user quotes and screenshots of Slack messages from happy engineers. Accepted.

The lesson: the elegant solution to the wrong problem is indistinguishable from the wrong solution.

2. What Y Combinator Actually Funds

"We would rather fund a team that builds a mediocre product in two weeks than a perfect product in six months."

Y Combinator Partner Wisdom, Repeated Across Office Hours

Y Combinator's acceptance rate has been estimated at roughly 1% by multiple independent analyses, with some recent cohorts dipping as low as 0.6%^{[7][1]}. That means 994 out of every 1,000 applications are rejected. If you want to be among the few who get in, you need to understand what separates them from the rest. The answer, drawn from analyzing the composition of recent cohorts and the stated preferences of YC partners, is not what most applicants think.

The Four Patterns of Accepted Founders

After studying published data on recent YC cohorts and partner commentary, four patterns emerge among accepted applications. None of them are about having the best idea or the most impressive technical approach.

Pattern One: Verified Customer Pain

Every accepted founder could point to specific people who had experienced the problem they were solving. Not hypothetically. Not "I think people struggle with this." They had names, phone numbers, and quotes. As Michael Seibel, Managing Director at Y Combinator, has emphasized repeatedly, the founders who succeed are those who can explain what they are building in two sentences plus a specific example^[8].

This specificity is not accidental. It is the result of deliberate customer discovery work before building. The founders who get in treat user conversations as the primary input to their product, not as a validation step after they have already decided what to build. YC partner Geoff Ralston has noted that "growth is the result of a great product, not the precursor"^[9].

Pattern Two: Demonstrated Execution Speed

YC partners have consistently emphasized that execution speed reveals founder quality more reliably than any other signal. Fast builders are decisive, pragmatic, and user-focused. Slow

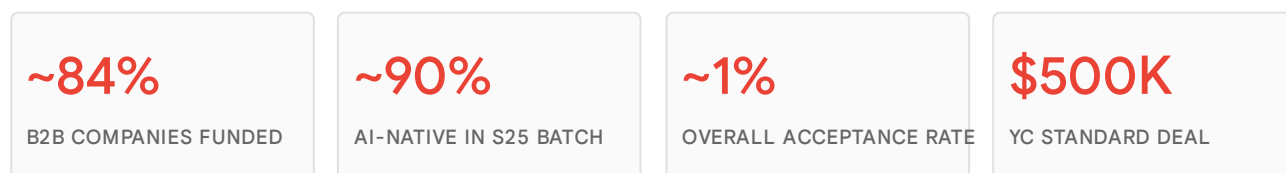
builders are perfectionists who optimize the wrong things. Paul Graham's landmark essay *Do Things That Don't Scale* remains one of the most cited pieces of startup advice precisely because it captures this ethos: the founders who win get their hands dirty, recruit users manually, and iterate rapidly based on real feedback^[10].

Pattern Three: Domain Depth, Not Domain Breadth

The funded startups of recent cohorts share a critical characteristic: they went deep into specific verticals rather than building horizontal platforms. Analysis of the Summer 2025 batch found that approximately 84% of funded companies were B2B, and nearly 90% explicitly incorporated AI into their core offering^[11]. A startup that automates FDA compliance document drafting for pharmaceutical companies wins against a startup that "uses AI to automate document generation for businesses." The narrow framing signals expertise, focus, and the ability to win a specific market before expanding.

Pattern Four: The Surprising Thing

YC's application asks, "Please tell us about something surprising or amusing that one of you has discovered." Most applicants skip this or treat it as an afterthought. The founders who get in use this question to demonstrate resourcefulness, unconventional thinking, or evidence that they ship against odds. Sam Altman, in his Stanford lecture series on startups, noted that the best founders have an "I always figure it out" mentality combined with the ability to attract people to their vision^[12].



The Y Combinator Standard Deal

In January 2022, Y Combinator revised its standard deal to \$500,000 total investment per company: \$125,000 for 7% equity plus an additional \$375,000 on an uncapped SAFE with Most Favored Nation provisions^[13]. This structure gives founders immediate capital to focus on building while deferring valuation negotiations until the next funding round. As Jared Heyman, a prolific YC ecosystem investor, noted, the deal "allows new startups to either destress fundraising or focus on growth to create FOMO on Demo Day"^[14].

Beyond the capital, YC provides access to a network of over 5,000 funded companies and 100+ unicorns, weekly office hours with partners who have collectively seen tens of thousands of startups, and the concentrated pressure of Demo Day preparation. The money is helpful but secondary. The network, the accountability, and the feedback density are what create outcomes.

The Myth of the Perfect Idea

A misconception that destroys more applications than any other is the belief that YC is looking for a perfect, original idea. They are not. They are looking for founders who can take a good idea and execute it with extraordinary speed and user focus. Multiple companies in every batch solve similar problems. YC has funded dozens of billing automation startups, countless developer tools, and innumerable AI infrastructure companies. The idea is not the differentiator. The team is.

This is liberating. You do not need to invent a new category. You need to find a painful problem in a proven market and solve it better than existing solutions because you understand your users more deeply. Michael Seibel has observed that "it's better to have 10 people loving your product than 1,000 people kind of liking it"^[15]. Your competitive advantage is not novelty. It is execution velocity rooted in customer intimacy.

THE IDEA DOES NOT MATTER (THAT MUCH)

YC has funded founders who pivoted multiple times during the batch. The initial idea was just the entry ticket. What mattered was the team's ability to learn, ship, and adapt. Do not spend weeks searching for the perfect idea. Spend days validating a good idea, then start building.

3. The AI-Native Distinction

"AI-native does not mean 'using AI.' It means the product could not exist without AI. The technology is not a feature. It is the foundation."

Industry Consensus, 2024-2025

There is a profound difference between a product that uses AI and a product that is AI-native. Most applicants get this wrong, and it costs them their credibility with YC partners. Understanding the distinction is not academic. It determines your architecture, your go-to-market, your competitive moat, and how you describe yourself in your application.

AI-Adjacent versus AI-Native

An **AI-adjacent** product takes an existing workflow and adds AI as a feature. A traditional project management tool that adds an "AI assistant" to help write task descriptions is AI-adjacent. Remove the AI, and the product still works. The AI is a convenience layer, not a structural element.

An **AI-native** product is built around the assumption that AI performs work that humans previously did. An AI agent that autonomously handles insurance claim follow-up calls is AI-native. Without the AI, there is no product. The AI is not a feature. It is the worker.

Table 1. AI-Adjacent versus AI-Native

Dimension	AI-Adjacent	AI-Native
Core value proposition	"We help you do X faster"	"We do X for you completely"
User relationship	Tool-assisted human work	Human-supervised agent work
Pricing model	Per-seat SaaS (\$20-50/user)	Per-outcome or value-based (\$500-5,000)
Competitive defense	Feature parity race	Workflow integration + data moat

YC framing

"AI-powered [category]"

"AI [job title] for [industry]"

The Worker, Not the Wrench

The most powerful mental model for AI-native products is this: your AI is not a tool that helps the user work. It is a worker that does the work, with the user supervising. This reframing changes your entire product design.

When your AI is a worker, the interface changes from "dashboard with AI features" to "supervisor cockpit." The user is not clicking through features. They are reviewing work, setting parameters, and intervening when the agent needs help. The product becomes a management layer for an autonomous workforce. Research on LLM-based multi-agent systems for software engineering confirms that hierarchical architectures with specialized agents significantly outperform monolithic approaches on complex tasks^[16].

This is why the multi-agent systems approach described later in this book is so powerful. You are not building a product with AI. You are building a system of AI workers managed by a human supervisor. The product is the coordination layer, not the execution layer. The ALMAS framework at JPMorgan demonstrated this practically by developing specialized agents for sprint planning, code generation, code review, and augmentation, successfully building a Python Streamlit application through autonomous multi-agent collaboration^[17].

THE WORKER FRAMING IN PRACTICE

Compare two pitches for the same underlying technology:

Tool framing: "Our AI-powered platform helps customer support teams draft responses faster using natural language generation."

Worker framing: "Our AI agent handles tier-one support tickets autonomously. It researches the issue, drafts the response, and sends it. Your human agents only review escalations. One agent now does the work of ten."

The second framing communicates a 10x improvement, not a 20% efficiency gain. It positions the product as replacing labor, not augmenting it. This is the difference between a \$50/month tool and a \$5,000/month worker replacement.

Why AI-Native Wins with YC

In the current application cycle, AI-native startups have a structural advantage with YC for several reasons. First, the timing aligns with YC's stated thesis. The organization has been explicit about its interest in AI and agentic systems. Second, AI-native companies can demonstrate traction faster because the value proposition is extreme: complete automation beats incremental improvement in user willingness-to-pay. Third, the market is expanding rapidly as enterprises realize that AI agents can replace entire job functions, not just assist them.

But this advantage is temporal. As AI capabilities commoditize, the distinction between AI-adjacent and AI-native will blur. The founders who win will be those who used this window to build data moats and workflow integrations that survive the technology transition. The AI is your entry ticket. Your understanding of the user's problem is your staying power.

THE COMMODITIZATION CLOCK

Every AI-native founder must answer this question: "What happens when the next foundation model can do what my custom agents do?" If your answer is "my users will just use the new model directly," you have no business. If your answer is "my workflow integration, domain data, and user-specific configurations make switching impossibly painful," you have a defensible company. Build the second answer.

PART II

The Problem

Finding something worth your life

4. Finding a Problem Worth Solving

"The best problems are the ones that make people angry. Not annoyed. Angry. Because anger means they have tried to solve it and failed."

Customer Discovery Heuristic, Repeated in YC Office Hours

The single highest-leverage decision you will make as a founder is which problem to solve. Everything else, your technology, your team, your funding, your eventual success or failure, flows from this choice. Yet most founders spend less time selecting their problem than they spend selecting their tech stack. This chapter is about correcting that imbalance.

The Problem Filter

Not every problem is a startup. Not every startup is a YC company. To find the intersection of "problem worth solving" and "problem YC will fund," apply these filters in order. Do not proceed to the next filter until the current one is satisfied.

Filter One: The Problem Is Experienced Daily

The best problems are frequent. A problem that occurs once per quarter does not create enough urgency for users to adopt a new solution, no matter how painful the quarterly event. Daily problems create daily habits. Daily habits create retention. Retention creates the metrics YC wants to see. Steve Blank, in his foundational work on customer development, emphasized that founders must get out of the building and validate assumptions through direct customer contact before writing code^[6].

Filter Two: The Problem Costs Real Money

There are two ways a problem costs money: directly, through waste or inefficiency, or indirectly, through opportunity cost. Direct costs are easier to quantify and sell against. "This problem costs your company \$10,000 per month in wasted labor" is a much stronger pitch than "this problem makes your employees unhappy." During customer discovery, always ask about cost. "If you had to put a dollar figure on this problem, what would it be?" The answer does not need to be precise. An order-of-magnitude estimate is enough.

Filter Three: Current Solutions Are Hated

The ideal problem already has attempted solutions that users actively dislike. This means the market is validated (people are paying for solutions) but the incumbent is vulnerable (people are unhappy). Your job is not to convince people to spend money solving a new category of problem. Your job is to convince them that your solution is dramatically better than what they suffer through today. As Paul Graham noted, great startup ideas are usually ones that seem bad to most people because the obvious ideas are already taken^[2].

Filter Four: You Can Solve It in Two Weeks

This is the YC constraint. You need a problem whose core solution can be built and demonstrated in two weeks. Not the complete product. Not the enterprise version. The core value proposition. If the minimum viable solution requires six months of development, the problem is too large for the YC timeline. You can always expand after proving the core value. But you cannot expand if you never ship because the first version was too ambitious.

Filter Five: You Have Access to Users

The best problem in the world is worthless if you cannot reach the people who have it. Before committing to a problem, verify that you can identify and contact at least fifty potential users. If your target user is "enterprise CFOs at Fortune 500 companies" and you know zero CFOs, you have an access problem. If your target user is "freelance graphic designers on Upwork" and you can message fifty of them today, you have a channel.

THE FIVE-FILTER SCORECARD

For each problem you are considering, rate it 1-5 on each filter. Minimum viable score: 20/25. If a problem scores below 4 on any single filter, eliminate it regardless of the total score. A problem with no accessible users is not a startup. It is a fantasy.

B2B versus B2C: The Data Speaks

Based on analysis of recent YC cohorts, approximately 84% of funded startups are B2B^[11]. This is not an accident. B2B startups have structural advantages that make them more fundable:

B2B buyers are trained to pay for value. A business will spend \$500 per month to save an employee ten hours. A consumer will not spend \$5 per month for anything that requires

explanation. The B2B sales cycle, while longer, produces higher lifetime values and more predictable revenue.

B2B problems are specific and measurable. "Reduce customer churn by 15%" is a concrete value proposition. "Help people discover music they will love" is not. Specificity makes selling easier, pricing clearer, and success demonstrable.

B2B growth compounds through relationships. One satisfied business customer can introduce you to their entire network. One enterprise deal can validate your product for an entire industry. B2C growth requires mass marketing. B2B growth requires credibility.

This does not mean B2C is impossible. It means B2C requires a different playbook, different metrics, and a higher tolerance for ambiguity. For your first YC application, B2B gives you the highest probability of success.

5. Customer Discovery for Builders

"People stop lying when you ask them for money."

Rob Fitzpatrick, *The Mom Test* (2013)

Customer discovery is not market research. Market research is reading reports and analyzing trends. Customer discovery is having conversations with real people about their real problems. It is uncomfortable, time-consuming, and absolutely non-negotiable. This chapter is a field guide to doing it right, grounded in the methodology developed by Steve Blank and refined by Rob Fitzpatrick.

The Discovery Mindset

The most common mistake founders make in customer discovery is treating conversations as sales opportunities. They pitch their idea, gauge reactions, and count positive responses as validation. This approach produces false positives because people are polite. They will tell you your idea is "interesting" right up until you ask them to pay for it.

The correct mindset is anthropological, not commercial. You are not selling. You are studying. Your goal is to understand the problem so deeply that you can describe it better than the person experiencing it. Rob Fitzpatrick termed this approach "The Mom Test" because it focuses on asking questions that even your mother cannot lie about: questions about her life and behavior, not about your idea^[5].

The Discovery Script

Here is a script framework adapted from Fitzpatrick's methodology and refined through practice:

Opening (establish context): "I am researching how [type of professional] handles [workflow area]. I am not selling anything. I am trying to understand if [specific problem] is actually a significant issue or if I am imagining it. Can you tell me about the last time you dealt with this?"

Probing (go deep): "How often does this happen? Walk me through exactly what you do step by step. How long does that take? What happens if it goes wrong? Who else is affected?"

Context (understand the ecosystem): "What tools do you use for this today? How did you choose them? What do you wish they did differently? If you could wave a magic wand, what would change about this process?"

Closing (test willingness to pay): "If I built something that [specific outcome] in [specific time], would you be open to trying it? What would you need to see to pay \$[price]/month for it?"

The closing question is the most important. It separates real problems from phantom problems. Fitzpatrick observed that "long story short, that person is a complainer, not a customer" when people talk enthusiastically about problems but never take steps to solve them^[5].

THE VALIDATION THRESHOLD

You need a minimum of twenty customer conversations to validate a problem. Not five. Not ten. Twenty. Among those twenty, at least five should express genuine frustration (not polite interest) and at least three should be willing to pay or pilot a solution before it exists. If you cannot find these numbers, your problem is not painful enough.

Finding People to Talk To

The most common excuse for skipping customer discovery is "I don't know anyone in that industry." This is almost never true. It is a discomfort avoidance mechanism. Here are practical channels that work:

LinkedIn direct outreach. Send ten personalized connection requests per day. Reference something specific from their profile. "I saw you manage billing at Regional Medical. I am researching how billing teams handle Medicare claim follow-ups. Would you have ten minutes for a quick call? No pitch, just learning." Expect a 10-20% response rate. Send fifty requests. Get five conversations.

Industry communities. Every industry has online communities. Subreddits, Discord servers, Facebook groups, Slack workspaces. Join them. Participate genuinely for a week. Then ask questions. The responses will be more honest than any formal interview because they are public and peer-validated.

Your existing network. The "friend of a friend" chain is more powerful than you think. Post on your personal social media: "I am researching [problem area]. Do you know anyone who works in [industry]? I would love to ask them ten minutes of questions."

Documenting What You Learn

Every conversation must be documented immediately after it ends. Memory degrades within hours. Your notes should include verbatim quotes, specific numbers, current workflow descriptions, current tools and costs, and willingness signals. Did they ask when your solution would be ready? Did they offer to introduce you to colleagues? Did they express frustration with current solutions?

After twenty conversations, compile a one-page problem thesis. This document becomes the foundation of your YC application, your investor pitch, and your product roadmap. It is the most important document you will create.

6. The Problem Thesis

"If you cannot explain the problem in one sentence that makes a stranger nod, you do not understand it well enough to build a solution."

Startup Communication Principle, Repeated Across YC Office Hours

After twenty customer conversations, you will have pages of notes, dozens of quotes, and a head full of insights. The problem thesis distills all of this into a single document that drives every subsequent decision. It is your compass. When you are tempted to add a feature, the thesis tells you whether it serves the core problem. When you are writing your YC application, the thesis provides the evidence. When you are pitching investors, the thesis becomes your narrative foundation.

The Thesis Structure

A problem thesis is one page. Not two. One. The constraint forces clarity. Here is the structure:

1. The Problem Statement (2-3 sentences). Describe the problem as experienced by a specific person in a specific context. Use the language your interview subjects used. Not your interpretation. Their words.

Example: "Medical billing specialists at mid-size hospitals spend 4-6 hours every Monday manually following up on denied insurance claims. Each follow-up requires navigating a different payer portal, re-submitting documentation, and waiting on hold for 20-30 minutes per call. A single hospital billing department processes 200-400 denied claims weekly, and current software only tracks claims but does not handle the follow-up workflow."

2. The Cost (quantified). Translate the problem into dollars, hours, or outcomes. Use numbers from your interviews.

Example: "A 200-bed hospital employs 6 billing specialists at \$55K/year each. Denied claim follow-up consumes 30% of their time, equivalent to \$99K in annual labor cost per hospital. Nationally, denied claims cost healthcare providers \$262 billion annually, with 63% of denials being recoverable through proper follow-up."

3. Current Solutions (and why they fail). List what people use today and the specific gaps your interviews revealed.

Example: "Existing solutions focus on claim submission and tracking but require manual follow-up. Billing teams use spreadsheets and shared email inboxes to manage the follow-up process. These tools fail because they do not automate the actual follow-up calls, portal submissions, and documentation resubmission."

4. The Opportunity (market size). Bottom-up calculation based on your specific user and price point.

Example: "6,129 hospitals in the US. Average spend on billing technology: \$180K/year. Addressable market: \$1.1B. Initial target: 200-bed hospitals (4,200 institutions). At \$3,600/month per hospital: \$181M serviceable market."

5. Customer Validation Evidence. Three direct quotes from interviews, with role and company type.

Example: "'Monday mornings are hell. I have forty denied claims and I know I will only get through ten.' - Billing Specialist, 150-bed community hospital. 'We have tried three different systems. None of them handle the follow-up.' - Billing Manager, regional health system."

THE THESIS TEST

Show your problem thesis to someone who knows nothing about your industry. If they understand the problem, its cost, and why current solutions fail within sixty seconds, your thesis is clear. If they ask clarifying questions, revise. Clarity is not an aesthetic choice. It is a competitive advantage.

From Thesis to Product Definition

The thesis tells you what problem to solve. It does not tell you how to solve it. The next step is defining the minimum product that delivers the core value. This is where most founders overbuild.

Define your product by working backward from the desired outcome. What is the one thing your product must do to make a user say "this is worth paying for?" Not the ten things. The one thing. Everything else is a distraction for the first version. Write this definition in one sentence. Post it where you can see it while building. When you are tempted to add a feature, ask: "Does this serve the one sentence?" If not, add it to a future list and stay focused.

PART III

The Build

Shipping what matters, fast

7. The Multi-Agent Workforce

"LLM-based multi-agent frameworks have great potential for automated software development, yet they suffer from misalignment between agents."

RTADev, ACL Findings (2025)

This chapter is the technical core of the book. It describes how to build software using a multi-agent system (MAS) as your development workforce. This approach enables a solo founder or small team to achieve output velocities that would normally require three to four engineers. The goal is not to replace human judgment. It is to amplify human direction.

I want to be clear about what this approach costs. API calls are not free. Expect to spend \$150-300 over a six-week build period on LLM inference. This is not zero cost, but it is approximately one hundred times cheaper than contracting a development team and approximately four hundred times cheaper than hiring a full-time engineer. The framing is not "free development." It is "radically cost-efficient development at extreme velocity."

Why Multi-Agent Systems Work for Product Development

A single large language model can write code. But software development is not just writing code. It is planning, implementing, reviewing, testing, debugging, and integrating. Each of these activities requires different skills, different contexts, and different evaluation criteria. A multi-agent system separates these concerns, creating specialized workers that collaborate under human supervision.

Research published in *ACM Transactions on Software Engineering and Methodology* confirms that LLM-based multi-agent systems significantly advance software engineering tasks through parallelization, specialization, and iterative refinement^[16]. The ALMAS framework at JPMorgan demonstrated this practically by developing specialized agents for sprint planning, code generation, code review, and augmentation, successfully building a Python Streamlit application through autonomous multi-agent collaboration^[17].

More practically, the parallelization is what matters. While you are sleeping, your agents can be writing tests. While you are talking to customers, your agents can be refactoring code. While you

are eating dinner, your agents can be resolving merge conflicts. The cumulative effect is compounding velocity that no solo human can match.

The Architecture: Hierarchical Supervision

The architecture follows four layers, each with distinct responsibilities. This is not theoretical. It is the architecture that has been used and refined through multiple product builds.

Layer One: Orchestration

The Orchestrator Agent receives high-level requirements from you and decomposes them into specific, assignable tasks. It maintains awareness of the project state, tracks dependencies between tasks, and routes work to the appropriate domain supervisor. The Orchestrator is the project manager, not the worker.

The Orchestrator's critical function is maintaining context across the project. When you say "add user authentication to the billing dashboard," the Orchestrator knows which files exist, which technologies are in use, and which other features might be affected. It translates your human intent into structured task definitions that worker agents can execute.

Layer Two: Domain Supervisors

Three domain supervisors manage specialized worker pools. The **Frontend Supervisor** manages all client-side code. The **Backend Supervisor** manages all server-side code. The **QA Supervisor** manages testing and quality assurance. Research by He, Treude, and Lo found that hierarchical supervision in multi-agent software engineering systems improves both output quality and coordination efficiency compared to flat agent architectures^[16].

Layer Three: Worker Agents

Each supervisor dispatches tasks to specialized workers. The Frontend Supervisor sends component generation tasks to a UI Worker, API integration tasks to a Client Worker, and styling tasks to a CSS Worker. The Backend Supervisor sends endpoint design to an API Worker, database queries to a Data Worker, and business logic to a Logic Worker. This granularity matters because each worker maintains a focused context. A UI Worker knows React patterns, accessibility standards, and component design. It does not need to know about database indexing or API rate limiting.

Layer Four: Tool Integration

All agents interact with external tools through standardized interfaces. The Git Worker commits code, creates branches, and manages pull requests. The LLM Worker handles all model inference, managing context windows and token budgets. The Deploy Worker manages staging and production deployments. The Monitor Worker tracks errors and performance metrics.

Agent Communication Protocol

Agents communicate through structured messages, not free-form conversation. This prevents the confusion and hallucination that occur when agents try to interpret natural language from other agents. The RTADev framework demonstrated that real-time alignment mechanisms between agents significantly improve functional completeness in software development tasks, achieving higher success rates than baseline multi-agent approaches^[18].

Every message contains: a unique identifier, sender and receiver addresses, a message type (task assignment, task result, review request, escalation), a structured task or result object, and relevant context references. This structure ensures that every agent knows exactly what is being asked, who is responsible, and what the acceptance criteria are.

The Human-in-the-Loop Framework

The most important architectural decision is where humans intervene. Too much human oversight and you lose the velocity advantage. Too little and agents ship broken code that destroys user trust. I recommend five human gates: architecture decisions (human approval required), code review (every PR reviewed), deployment approval (staging automatic, production manual), feature completion (human verifies on staging), and escalation (when agent confidence drops below 80%).

Total daily human time: approximately 90 minutes. If you are spending more than this, your agent prompts or architecture needs refinement.

THE TEMPTATION TO MICROMANAGE

New MAS operators constantly check agent output, second-guess decisions, and intervene prematurely. Resist this. Set your gates, trust the system, and focus your freed time on customer conversations and product strategy. The whole point of agents is to give you that time.

Harness Engineering: Why It Matters

There is an important distinction in how you supervise your multi-agent system. The approach described throughout this book is **harness engineering**: you design the harness, define the rules, set the gates, and let the agents run within those constraints. You are not micromanaging each decision. You are architecting the system within which agents make thousands of decisions autonomously.

The harness is your competitive advantage. Two founders using the same LLM API with the same agent framework will produce radically different outputs based on the quality of their harness. The harness includes: your prompt engineering, your agent role definitions, your communication protocol, your quality gates, your error handling strategy, and your escalation rules. These are not default configurations. They are the product of iteration, debugging, and deep understanding of your specific build process.

Invest time in refining your harness. The first week of using a MAS should produce a detailed runbook: which prompts work, which fail, where agents get confused, and how you corrected them. This runbook is your intellectual property. It is why another founder cannot replicate your output speed simply by copying your tech stack.

A Note on Loop Engineering

You may encounter discussions of **loop engineering** in research literature and advanced multi-agent frameworks. Loop engineering refers to systems where agents run in continuous feedback loops, autonomously refining their output through dozens or hundreds of self-correcting iterations. While theoretically powerful, these systems consume enormous amounts of tokens. A single loop engineering task can burn through \$50-100 in API costs, with no guarantee of convergence.

For early-stage founders working within a \$150-300 six-week budget, loop engineering is not practical. The token consumption is simply too high for the uncertain payoff. The harness

engineering approach described in this book uses single-pass execution with human review gates. It is less elegant than continuous self-improvement, but it ships code you can afford. When you have raised funding and have a larger inference budget, loop engineering becomes worth exploring. Until then, build the harness, ship the product, and talk to users.

8. Architecture for Speed

"Choose the stack that lets you ship tomorrow, not the stack that would impress at a conference talk."

Pragmatic Engineering Principle, Echoed Across YC Batches

Your technology stack is a means to an end. The end is shipping a product that solves a real problem for real people. Every technical decision should be evaluated against one criterion: does this get me to a user-validated product faster? If the answer is not an unambiguous yes, choose the simpler option.

The Zero-Cost Stack

This stack is chosen for a specific constraint: maximum capability at minimum cost. Every service listed has a free tier sufficient for building and launching an MVP. Total expected cost over six weeks: \$150-300, primarily for LLM API calls.

Table 2. Recommended Zero-Cost Stack

Layer	Choice	Free Tier	Rationale
MAS Framework	CrewAI	Open source	Fastest agent framework. Production-ready. Role-based abstractions.
LLM Access	OpenRouter + GPT-4o-mini	\$5-10/day	Single API for multiple models. Automatic fallback if one fails.
Frontend	React + Vite + Tailwind	Open source	Fast dev cycle. Agents write React well. Tailwind reduces CSS complexity.
Backend	FastAPI (Python)	Open source	Python ecosystem for ML/AI. Agents write Python natively. Async by default.
Database	Supabase PostgreSQL	500MB	Database + Auth + Storage + Realtime in one platform.

Authentication	Clerk	10,000 users	Pre-built React components. Social login. Session management.
Hosting	Vercel (FE) + Render (BE)	Hobby tier	Auto-deploy from Git. Preview deployments for every PR.
Payments	Stripe	Pay per transaction	Industry standard. Agents understand Stripe's API patterns.
Monitoring	Sentry	5,000 events	Error tracking with context. Sufficient for MVP stage.
Repository	GitHub + Actions	Unlimited public	Agents commit code. Actions run tests and deploy.

Why This Stack Over Alternatives

Agent familiarity: Large language models have been trained on enormous quantities of React, Python, FastAPI, and Tailwind code. They write these technologies fluently. They struggle with newer or more obscure frameworks. By choosing popular, well-documented technologies, you get better agent output with less prompting effort.

Community depth: When something breaks at 2 AM (and it will), you need Stack Overflow answers, GitHub issues, and Discord communities. This stack has them in abundance. Obscure technologies have beautiful documentation and zero community support.

Free tier sufficiency: Every service in this stack offers a free tier that handles an MVP and its first users. You will not hit a paywall during the critical first six weeks. When you do eventually need to pay, you will have revenue to cover it.

THE SIMPLICITY TEST

If you cannot explain your architecture to a non-technical customer in thirty seconds, it is too complex. Simplify until you can. Complex architectures are a form of procrastination masquerading as preparation.

9. From Code to Customers

"Startups can only solve one problem well at any given time."

Y Combinator Essential Startup Advice

You have built the product. You have orchestrated agents, reviewed code, deployed to production, and verified that everything works. This is an achievement. It is also the easy part. The hard part is getting people to use it, trust it, and pay for it. This chapter is about crossing that chasm.

The Pilot Program Strategy

Do not launch to the world. Launch to five specific people. A pilot program is a controlled experiment with human participants who understand they are early users. This framing gives you permission to be imperfect while creating the conditions for genuine feedback and eventual conversion.

Structure your pilot program explicitly: two weeks duration, specific user commitments (use the product three times and provide feedback in a 15-minute call), personal onboarding via video call where you watch them sign up and complete their first task, and a conversion path where you ask for payment at the end. This is the ultimate validation. If someone who has used your product for two weeks will not pay for it, your product is not solving a painful enough problem.

The Pricing Conversation

Most first-time founders price too low. They want to remove friction. They want users to say yes. But a low price signals low value. If your product saves someone ten hours per week, charge accordingly.

Value-based, not cost-based. Your price should reflect the value you create, not your costs. If your agent handles \$50,000 worth of manual work annually, charging \$500 per month is a 10x return for the customer. That is an easy sell.

Anchor against labor costs. Frame your price against the cost of human labor. "One billing specialist costs \$4,500 per month. Our agent does 80% of their work for \$900 per month." This

makes your price feel small, not large.

Start high, discount for pilots. Set your standard price higher than you think. Offer pilot users 50% off for the first three months. This frames the discount as a favor, not the base price. When the discount expires, the user has already integrated your product into their workflow.

Growth Mechanics for the First Hundred Users

Before you have product-market fit, growth is manual. There are no growth hacks, no viral loops, no marketing automation that substitutes for founder hustle. The Stripe founders became famous within YC for the "Collison Installation": when anyone agreed to try Stripe, they would say "Right then, give me your laptop" and set them up on the spot^[10]. Airbnb's founders went door-to-door in New York recruiting users and helping them improve their listings. Pinterest's founder recruited initial users by talking to strangers in coffee shops.

Here are the tactics that work: direct outreach at scale (twenty personalized messages per day), community participation (join every community where your users gather and help genuinely), case study content (your first paying customer is your most valuable marketing asset), and referral incentives (offer one free month for every paying customer referred).

THE TRACTION THRESHOLD

By the time you submit your YC application, you need at least one of these: \$500+ monthly recurring revenue, 10+ weekly active users, or 3+ paying customers. If you do not have one of these, do not apply yet. Spend another week on acquisition. Applications without traction have a near-zero acceptance rate regardless of how elegant the idea or architecture.

PART IV

The Pitch

Telling the story that wins

10. The Y Combinator Application

"YC partners review thousands of applications. Clarity isn't a nice-to-have; it's the minimum bar. If your description requires a paragraph of context before it makes sense, it won't survive the first filter."

Y Combinator Application Guidance (2025)

Your YC application will be read in under five minutes by a partner who has read thousands of similar applications. Every word must earn its place. Every claim must be grounded in evidence. Every answer must reveal the kind of founder who cannot be stopped. This chapter is about crafting an application that demands attention.

The Philosophy of the Application

The YC application is not a test you pass. It is a story you tell. The story is about a person who discovered a painful problem, built a solution despite constraints, found users who wanted it, and now needs capital and network to scale. Your job is to make this story so compelling that the partner reading it wants to be part of it.

The narrative arc is simple but non-negotiable: you lived the problem; you validated it with real people; you built something despite constraints; people are using it and paying for it; you are the right person to scale this. If any element of this arc is missing, the story collapses.

Field-by-Field Guide

"What is your company going to make?"

This is the most important field. Spend 40% of your application time on this one answer. Michael Seibel advises that you need a simple two-sentence explanation of what your company does, plus a specific example. Do not use jargon. Explain it in simple terms anyone would understand^[8].

Bad: "We are building an AI-powered platform that leverages multi-agent systems to revolutionize workflow automation for businesses through intelligent process orchestration."

Good: "ClaimPilot is an AI agent that handles denied insurance claim follow-ups for hospital billing departments. Unlike Waystar, which only tracks claims, our agent actually makes the follow-up calls, navigates payer portals, and resubmits documentation. Regional Medical reduced their billing team's Monday workload from six hours to forty-five minutes."

"Why did you pick this idea to work on?"

This question tests founder-market fit. The answer must be personal and specific. Connect your history to the problem domain.

Bad: "We saw a huge opportunity in the healthcare billing market and believe AI can transform the industry."

Good: "I spent two years as a billing coordinator at a 200-bed hospital. Every Monday, I processed forty to sixty denied claims manually. I tried Waystar, Experian Health, and even a custom Excel system. None handled the actual follow-up. When I left, I called five former colleagues and every single one said the same thing: 'If you build this, we will buy it on day one.' I am building this because I lived the problem and I know exactly how to solve it."

"What's new about what you're making?"

Be specific about your mechanism, not your category. Not "we use AI agents." Instead, explain the specific domain knowledge or workflow integration that makes your approach different.

"How do or will you make money?"

Show you have thought about business model and unit economics. Specific pricing, comparison to current costs, and revenue potential at scale.

"How will you get users?"

Demonstrate that you already have a working acquisition channel. Documented channels with conversion rates. "We found our first five customers through direct LinkedIn outreach to billing managers. Our response rate is 12%, meeting booking rate is 30%, and pilot-to-paid conversion is 60%."

THE DATA IMPERATIVE

Notice what every good answer has in common: specific numbers, named people, and documented outcomes. Vague claims signal vague thinking. Precision signals competence. When in doubt, add a number. When uncertain, name a person.

The One-Minute Video

The video is not a production. It is a proof of founder quality. Partners watch at double speed. You have thirty seconds of actual attention. The script structure:

Table 3. Video Script by Seconds

Time	Purpose	Example
0-5s	Name + hook	"I am [Name], founder of [Company]."
5-15s	Personal problem	"I spent two years manually following up on denied insurance claims. It took six hours every Monday."
15-30s	Solution demo	"So I built [Product]." [Screen recording] "Our AI agent handles the entire follow-up process."
30-45s	Traction	"Three hospitals are using it. One reduced their Monday billing time by 85%. We are at \$1,800 MRR after six weeks."
45-55s	Market + ambition	"Medical billing is a \$28B market. We start with follow-ups, then automate the entire revenue cycle."
55-60s	Close	"I am a solo founder with AI agents for a team. YC will help us go from three hospitals to three hundred."

Technical requirements: 1080p minimum, face a window for lighting, use a USB microphone for clear audio, keep it to exactly 60 seconds, and show your product actually working.

11. The Interview and Beyond

"YC interviews are 10 minutes with 2-4 partners. Rapid-fire, pressure-tested, no preamble."

Y Combinator Interview Process Documentation

If your application passes the initial review, you will be invited to a ten-minute video interview with two to four YC partners. This is the fastest, most high-stakes conversation of your founder career. Ten minutes to demonstrate that you are competent, committed, and capable of building something massive. Of every 1,000 applications, roughly 60 to 100 get interviews, and 15 to 20 get accepted^[7]. This chapter is about winning that conversation.

What Partners Are Testing

The interview is not about your product. It is about you. Partners have already read your application. They know what you are building. What they do not know is whether you are the kind of person who can execute under pressure, adapt to feedback, and build relentlessly for ten years.

Clarity of thought. Can you explain complex things simply? When asked a hard question, do you think before speaking or ramble? Do you answer the question that was asked, or the question you wish had been asked?

Speed of learning. When a partner challenges your assumption, do you defend or do you consider? Can you incorporate new information in real time?

Relentlessness. Do you have the energy and commitment to push through the inevitable hard times? Can you point to specific evidence that you ship despite obstacles?

Market insight. Do you understand your market at a granular level? Not market size reports. The actual dynamics of how decisions are made, who influences them, and why incumbents are vulnerable.

The Questions You Will Face

Table 4. Common Interview Questions and Strategy

Question	What They Want	Model Answer
"Why you?"	Founder-market fit	"I spent two years in medical billing. I know the forty-seven reasons claims get denied. An engineer without this experience would take a year to learn what I know."
"What if Google builds this?"	Defensibility analysis	"Google builds horizontal platforms, not vertical workflows. They will never encode Medicare billing rules. Our moat is domain depth, not technology."
"Why is this not just a feature?"	Market scope	"Follow-up calls are just the entry point. We are building the autonomous billing department, not a call automation tool."
"How do you know customers want this?"	Validation depth	Name three customers, quote their feedback, cite conversion metrics.
"What is your biggest weakness?"	Self-awareness	"Enterprise sales cycles. We close pilots in two weeks but converting to annual contracts takes six to eight weeks."

The Post-Interview Reality

After the interview, you will receive a decision within 24 hours. If accepted, you have until the batch start date to prepare. If rejected, you have two options: reapply for the next batch with improved traction, or continue building without YC. Rejection is not a verdict on your potential. It is a signal about your current stage. Many successful companies were rejected multiple times before acceptance. The correct response to rejection is demonstrated growth.

If accepted, the batch is approximately ten weeks of intense work. You will have weekly office hours with a partner, access to YC's network of alumni and investors, and the pressure of Demo Day at the end. The \$500,000 investment is helpful but secondary. The network, the accountability, and the concentrated feedback are what create the outcomes that make headlines.

PART V

The Reality

After the application, before the IPO

12. The First Hundred Days

"Being a founder is incredibly difficult, stressful, and requires 24 hour focus. Being a founder isn't glamorous."

Dustin Moskowitz, YC How to Start a Startup Lecture (2014)

Whether or not you get into YC, the first hundred days after your application submission are the most formative period of your company. This is when you either build momentum or lose it. When you either prove that your traction was not a fluke or discover that your initial users were not representative. This chapter is about navigating that period with discipline.

If You Get In

Congratulations. Now the real work begins. YC's value is not the money. It is the compression of learning that occurs when you are surrounded by other founders who are moving fast, when you have weekly accountability to a partner who has seen thousands of companies, and when you have a hard deadline called Demo Day that forces prioritization.

The most important habit during the batch is maintaining your customer conversation velocity. It is easy to get distracted by batch activities, social events, and networking. Do not. The founders who raise the most on Demo Day are the ones who kept talking to users while everyone else was attending mixers. As Sam Altman advised in his Stanford lecture series on startups, "Build products, talk to users, eat, sleep, and nothing else"^[12].

If You Do Not Get In

Rejection stings. Let it sting for one day. Then get back to work. YC is one path among many. The companies that matter are built by founders who would build regardless of whether an accelerator accepted them.

Your action plan after rejection: Week one, analyze honestly and identify the weakest element. Weeks two through four, execute the improvement plan. Weeks five through eight, continue building and growing with the same discipline you would have applied during the batch. Week

nine, reapply if the next batch deadline allows. YC openly encourages reapplication and has noted that many successful alumni were rejected on previous attempts.

The Metrics That Matter

Whether you are in YC or not, these metrics determine your company's trajectory: weekly active users (people who actively used your product this week, not registered users or downloads), revenue growth rate (20-30% month-over-month is the threshold that signals product-market fit), retention cohorts (of users who started in month one, what percentage are still active in month three), net promoter score (ask users how likely they are to recommend your product to a colleague; scores above 50 indicate strong product-market fit), and unit economics (customer acquisition cost, lifetime value, payback period).

13. The Founder's Discipline

"The very best startup ideas tend to have three things in common: they're something the founders themselves want, that they themselves can build, and that few others realize are worth doing."

Paul Graham, Y Combinator Co-Founder (2005)

This is the final chapter, and it is about the only thing that ultimately matters: the discipline of building for humans, not for portfolios. I have given you tactical advice on customer discovery, multi-agent architecture, application writing, and interview preparation. But tactics are worthless without the right foundation. This chapter is about the practices that separate founders who build lasting companies from those who build impressive demos.

Build for Humans, Not Portfolios

The most dangerous trap for technical founders is building to demonstrate capability rather than to serve users. The portfolio mindset produces elegant solutions to unimportant problems. It produces architectures that impress engineers and confuse customers. It produces demo videos that look great and products that no one uses.

The antidote is relentless user focus. Before every decision, ask: "How does this serve the person who is paying me?" Not "How does this showcase my technical sophistication?" The user does not care about your microservices architecture. They care that their problem is solved quickly, reliably, and affordably. Paul Graham put it directly: "You can't wait for users to come to you. You have to go out and get them"^[10].

This sounds obvious. It is not. The temptation to optimize for impressiveness is constant. Every technical blog post, every framework announcement, every AI research paper creates pressure to adopt the new and shiny. Resist. Your users do not want shiny. They want solved.

The Long Game

YC acceptance is not success. It is a milestone. The real journey is ten years of building, adapting, and persisting. The founders who create enduring companies are not the ones who had the best

YC applications. They are the ones who maintained their user obsession through every phase of growth.

In year one, you are discovering the problem. In year two, you are finding product-market fit. In year three, you are scaling what works. In year five, you are defending against competitors. In year ten, you are reinventing yourself to stay relevant. At every stage, the companies that win are the ones that never lost touch with the humans they serve.

Discipline in Practice

Every week, talk to at least one user. Not about your product. About their life, their work, their challenges. The conversations that produce the best product insights are the ones where your product is never mentioned.

Every month, do the job your product automates. If you build billing automation, spend a day doing billing manually. If you build code review tools, review code without your tool. The distance between your product and the actual work is where bad decisions live.

Every quarter, ask five users: "What would make you leave?" This question surfaces the weaknesses and gaps you are too close to see. It also demonstrates to users that you care about retention, not just acquisition.

Every year, revisit your problem thesis. Markets evolve. Problems shift. The thesis that was correct eighteen months ago may no longer be accurate. Update it based on new learning. Let it guide your pivot decisions.

THE FINAL CHECK

Before you submit your YC application, before you write your first line of code, before you even choose a problem, answer this question honestly: "If I never get funded, if I never go viral, if I build this quietly for five years with modest revenue, would I still want to do it?"

If the answer is yes, you have found your problem. Build it. Ship it. Serve your users. Everything else follows.

APPENDICES

Tools and References

For the journey ahead

A. The Six-Week Execution Checklist

Table 5. Week 1: Problem Validation

Task	Deliverable	Done
Identify 50 potential customers in target vertical	Contact list with names, roles, and contact info	
Send 30 personalized outreach messages	10+ responses scheduling calls	
Conduct 5 customer discovery calls	5 recorded transcripts	
Document 50+ verbatim pain quotes	Quote spreadsheet with source and context	
Write 1-page problem thesis	Validated thesis document	
Write 1-page competitive analysis	Competitor matrix with differentiation	

Table 6. Week 2: Build

Task	Deliverable	Done
Write 1-page product requirements document	PRD with core workflow defined	
Design MAS architecture	Architecture diagram + agent roles	
Set up development environment and repos	GitHub repos with CI/CD configured	
Implement Orchestrator + Supervisors	Working agent system	
Build core product functionality	End-to-end demo workflow	
Record 2-minute demo video	Demo video file	

Table 7. Week 3: First Users

Task	Deliverable	Done
Implement top 3 feedback items from pilot demos	Improved product	
Build user onboarding flow	60-second time-to-value	
Deploy to production	Live product URL	
Onboard 3 pilot customers	3 active users	
Document all friction points	Improvement priority list	

Table 8. Week 4: Traction

Task	Deliverable	Done
Integrate Stripe billing	Working payment flow	
Convert 2+ pilot users to paid	First revenue	
Cold outreach to 20 new prospects	20 personalized messages sent	
Post on relevant communities	3 posts with engagement	
Close 1-2 new paying customers	\$500+ MRR or 10+ active users	

Table 9. Week 5: Application

Task	Deliverable	Done
Draft YC application v1	Complete application draft	
Script 60-second video	Video script with timing	
Record and edit video	Final 60-second video	
Get feedback from 2 advisors	Feedback incorporated	

Final application review

Application ready to submit

Table 10. Week 6: Submit

Task	Deliverable	Done
Final proofread and link check	All fields verified	
Submit application	Confirmation received	
Continue growth (do not pause)	+2 users or +\$200 MRR	
Prepare interview answers	Written responses to 10 common questions	

B. MAS Prompt Library

These prompts initialize the core agents in your development system. Customize them for your specific product domain and technology choices.

Orchestrator Agent

You are the Orchestrator of a software development team.

PROJECT: [Your product name and one-sentence description]

STACK: React + FastAPI + PostgreSQL + Tailwind CSS

GOAL: [Current sprint goal]

Your responsibilities:

1. Receive requirements from the human founder and decompose into tasks
2. Route tasks to the appropriate Domain Supervisor (frontend, backend, qa)
3. Track dependencies between tasks and manage execution order
4. Report progress every 2 hours with completed tasks, blockers, and next steps
5. Escalate to human when: architectural decisions needed, repeated failures, confidence < 80%

Rules:

- Never write code. Delegate to supervisors.
- Always include acceptance criteria with task assignments
- Maintain awareness of project state
- Ask for clarification when requirements are ambiguous
- Prioritize speed over perfection for MVP features

Frontend Supervisor

You are the Frontend Supervisor for a React application using Tailwind CSS.

TECHNOLOGY: React 18, Vite, Tailwind CSS, React Query, React Router

PATTERNS: Functional components, custom hooks, container/presenter pattern

Your responsibilities:

1. Receive UI tasks from the Orchestrator and break them into component-level work
2. Assign tasks to UI Worker, Client Worker, or CSS Worker based on type
3. Review all frontend code for consistency, accessibility, and responsiveness
4. Ensure all components handle loading, error, and empty states

Rules:

- Mobile-first responsive design
- WCAG 2.1 AA accessibility compliance
- No inline styles. Use Tailwind utility classes.
- All API calls go through the Client Worker using React Query

Backend Supervisor

You are the Backend Supervisor for a FastAPI application using PostgreSQL.

TECHNOLOGY: Python 3.11, FastAPI, SQLAlchemy, Pydantic, Alembic

PATTERNS: Repository pattern, dependency injection, async endpoints

Your responsibilities:

1. Receive backend tasks from the Orchestrator and break them into endpoint/function work
2. Assign tasks to API Worker, Data Worker, or Logic Worker
3. Review all backend code for security, performance, and correctness
4. Ensure all endpoints have proper validation and error handling

Rules:

- All endpoints async with proper HTTP status codes
- Input validation with Pydantic models
- SQL injection prevention via SQLAlchemy ORM (no raw SQL)
- Authentication on all non-public endpoints

QA Supervisor

You are the QA Supervisor responsible for testing and quality assurance.

TECHNOLOGY: pytest, Playwright, coverage.py

TARGET: 80%+ code coverage, all critical paths tested

Your responsibilities:

1. Generate test cases from requirements and acceptance criteria
2. Write unit tests for business logic
3. Write integration tests for API endpoints
4. Write E2E tests for critical user flows
5. Perform security scanning (OWASP top 10 checks)

Rules:

- Test names must describe what is being tested
- Mock external dependencies (APIs, databases)
- Test edge cases and error paths, not just happy paths
- No code merges without passing tests

C. YC Application Field Reference

Table 11. Complete Application Field Guide

Field	Strategy
Company name	Use real domain you own
Company URL	Live product, not landing page
What is your company going to make?	One sentence. Specific user, specific outcome, specific differentiation.
Why did you pick this idea?	Personal connection + market validation
What's new about what you're making?	Specific mechanism, not "we use AI"
What do you understand that others don't?	Domain insight that took time to acquire
Who are your competitors?	3-5 real competitors + why they fail
How do/will you make money?	Specific pricing + unit economics
How will you get users?	Documented channels + conversion rates
Current status	Users, revenue, product stage with dates
Something surprising	Genuine surprise demonstrating resourcefulness

D. Idea Evaluation Matrix

Score each potential idea on these criteria. Minimum viable: 20/25.

Table 12. Idea Scoring Framework

Criterion	Weight	Score 1-5
Personal domain expertise	Critical	___
Market size (\$1B+)	Critical	___
Agentic build feasibility (2 weeks)	Critical	___
B2B focus	Important	___
YC Requests for Startups alignment	Important	___
Speed to first revenue (2 weeks)	Critical	___
Defensibility potential	Important	___
Total		___/35

References

- [1] Y Combinator. (n.d.). Resources for investors. Y Combinator. Retrieved from <https://www.ycombinator.com/investors>
- [2] Graham, P. (2012). How to get startup ideas. *PaulGraham.com*. Retrieved from <http://paulgraham.com/startupideas.html>
- [3] Lindgaard, G., Fernandes, G., Dudek, C., & Brown, J. (2006). Attention web designers: You have 50 milliseconds to make a good first impression! *Behaviour & Information Technology*, 25(2), 115-126. <https://doi.org/10.1080/01449290500330448>
- [4] Norman, D. A. (2013). *The design of everyday things: Revised and expanded edition*. Basic Books.
- [5] Fitzpatrick, R. (2013). *The mom test: How to talk to customers and learn if your business is a good idea when everyone is lying to you*. CreateSpace Independent Publishing Platform.
- [6] Blank, S. G. (2005). *The four steps to the epiphany: Successful strategies for products that win*. Cafepress.com.
- [7] We Are Founders. (2026, April 27). Y Combinator acceptance rate 2026: What the data shows. Retrieved from <https://www.wearefounders.uk/y-combinator-acceptance-rate-2026/>
- [8] Seibel, M. (n.d.). How to perfectly pitch your seed stage startup. Y Combinator. Retrieved from <https://fondo.com/blog/perfectly-pitch-seed-stage-startup-y-combinator-michael-seibel>
- [9] Y Combinator. (2020, June 9). YC's essential startup advice. Retrieved from <https://www.ycombinator.com/library/4D-yc-s-essential-startup-advice>
- [10] Graham, P. (2013). Do things that don't scale. *PaulGraham.com*. Retrieved from <http://paulgraham.com/ds.html>
- [11] We Are Founders. (2026, April 27). Y Combinator acceptance rate 2026: B2B dominates. Retrieved from <https://www.wearefounders.uk/y-combinator-acceptance-rate-2026/>
- [12] Altman, S. (2014). How to start a startup [Lecture series]. Stanford University. Retrieved from https://www.youtube.com/playlist?list=PL5q_lef6zVkaTY_cT1k7qFNF2TidHCE-1
- [13] Y Combinator. (2022, January 10). YC's \$500k standard deal. *Hacker News*. Retrieved from <https://news.ycombinator.com/item?id=29878365>
- [14] Heyman, J. (2022, January 12). On the \$500k Y Combinator standard deal. *Medium*. Retrieved from <https://jaredheyman.medium.com/on-the-500k-y-combinator-standard-deal-b45fd8bd867c>
- [15] Chen, S. (2022, March 13). Y Combinator Q&A at MIT: Start-up advice from Michael Seibel. *Medium*. Retrieved from <https://medium.com/@shenseanchen/y-combinator-q-a-mit-start-up-advice-from-michael-seibel-573fbfb657c5>

- [16] He, J., Treude, C., & Lo, D. (2025). LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology*, 34(5), Article 124. <https://doi.org/10.1145/3712003>
- [17] JPMorgan Chase. (2025). ALMAS: An autonomous LLM-based multi-agent software engineering framework. *arXiv preprint*. <https://arxiv.org/abs/2510.03463>
- [18] RTADev. (2025). Intention aligned multi-agent framework for software development. In *Findings of the Association for Computational Linguistics (ACL 2025)*. <https://aclanthology.org/2025.findings-acl.80.pdf>

Built for Humans © 2026 Sreekesari Vangeepuram

Licensed under CC BY-NC-SA 4.0

builtforhumans@proton.me

Build for people. The rest follows.